Design a Banking Ledger
Design a Review Tracker System
Design a Link Tree Search System
Design a Banking Interface
Design a Ticketing System
Design a Scheduling System
Design a Scheduling Notification System
Design a Tic-Tac-Toe Tester
Design a Blackjack Game
Design a Shopping Cart System
Design an File System
Design a Restaurant Management System

14
Design a Restaurant Management System

In this chapter, we will explore the design of a Restaurant Management System. The goal is to create classes that represent the various components, such as menus, reservations, and tables, the different actions a user can perform, and the interactions between the building blocks, manages orders, and assigns waiting staff. Ultimately, we aim to design a both straightforward and flexible for future enhancements.

Requirements Gathering

The first step in designing a Restaurant Management System is to clarify the requirements and define the scope. Here's an example of a typical conversation between a candidate and an interviewer:

"Anyone yourself planning a dinner using a Zoom call, I'm hoping that the restaurant is nearest to a table for your group, check available times, and send a link to your phone. Anyone who's the staff wants your job, I'm hoping that they can log in, see which tables need attention, and update the status. Anyone who's the system is smoothly handling table reservations, handling orders, and calculating costs. We're designing a Restaurant Management System for dine-in operations."

Requirements clarification

Here is an example of how a conversation between a candidate and an interviewer might unfold:

Candidate: Let's start by setting the scope. I assume the system manages reservations, menus, order tracking, and payments. For simplicity, I'll assume it's a single-location restaurant. Is that correct?
Interviewer: Yeah, a reasonable starting point.
Candidate: Should customers place orders directly on the system, or do they order through a waiter?
Interviewer: Yes, customers can place orders directly on a tablet and a waiter is also involved.
Candidate: How does the system determine if a table is available for a new reservation?
Interviewer: Each table has a fixed seating capacity. If a party of 4 wants a table, we need to find a free table that seats at least 4.
Candidate: Can a reservation be for more than one table? Or do we assume one table per reservation?
Interviewer: No, reservations cannot span multiple tables.
Candidate: When a party orders a meal which arrives, do they automatically get their meal and bill?
Interviewer: No, they must pay. The waiter will bring the bill, and the system needs to track payments, which already has a pending status.
Candidate: What happens when a table is ready and there's a waiting list? Do we notify the next party?
Interviewer: The system should assign a table and mark it as ready based on current availability and the waiting list.
Candidate: Does the system allow orders to be altered or removed after they're placed?
Interviewer: Yes, you can remove items or update them as long as the order hasn't been prepared.
Candidate: Does the system track the status of orders?
Interviewer: Yes, it should track the status of these orders.
Candidate: Are there rules for splitting the bill at the end?
Interviewer: For now, just assume everyone's single bill per party.

Requirements

Based on this questions and answers, we can note the functional requirements for our restaurant management system.

Core functionality

- Customers can book tables for a future date and time based on availability.
- Each reservation is associated with a party size.
- The system must assign a table to a reservation when the party arrives.
- The system assigns a table to a walk-in customer, and customers get it when they arrive using their name.
- Customers can cancel reservations.

Menu & ordering

- The system must display a menu to customers.
- Customers can place orders directly on a tablet.
- Orders can be modified or removed as long as they haven't been prepared.
- The system assigns a table to a walk-in customer, and customers get it when they arrive using their name.
- Customers can cancel reservations.

Table management

- The system allows orders to be altered or removed after they're placed.
- The system tracks the progress of orders.

Billing

- The system generates a single total bill amount per checkout.

Notes and Bonus Considerations

- The system should handle increased traffic during busy periods (e.g., weekend evenings) without performance degradation, maintaining concurrent operations.

With these requirements, we are ready to model the objects for the core system.

Identify Core Objects

Let's identify the core objects of the restaurant management system.

- Customers book tables for a future date and time based on availability.
- Each reservation is associated with a party size.
- The system must assign a table to a reservation when the party arrives.
- The system assigns a table to a walk-in customer, and customers get it when they arrive using their name.
- Customers can cancel reservations.

Menu & ordering

- The system must display a menu to customers.
- Customers can place orders directly on a tablet.
- Orders can be modified or removed as long as they haven't been prepared.
- The system tracks the progress of orders.

Billing

- The system generates a single total bill amount per checkout.

Notes and Bonus Considerations

- The system should handle increased traffic during busy periods (e.g., weekend evenings) without performance degradation, maintaining concurrent operations.

With these requirements, we are ready to model the objects for the core system.

Design Class Diagram

Now let's dive into the heart of the object-oriented design interview. Identifying classes and interfaces, diagring data and state flow, and ensuring clean, maintainable, and testing-ready code with clear responsibilities.

Below, we'll design each class, its properties, and its responsibilities, ensuring a clear separation of concerns.

Menu

The Menu class represents the restaurant's menu, storing menu items in a map with names as keys to quickly retrieve them for ordering. It exposes methods to add items, get items by name, and display the full menu. Keeping this in a dedicated class makes it easy to modify or extend the menu without affecting other parts of the system.

Below is the representation of the class.

MenuItem

The MenuItem defines each dish on the menu, holding its name, description, price, and category for use in orders. The Menu class stores these to provide the full menu, and the Order class references them when adding items to customer orders. The Category enum ensures valid menu categories, preventing invalid entries and enabling filtering by category if needed.

MenuItem class and Category enum

Table

The Table class models restaurant tables. Some of the attributes, like capacity and isAdmin, rarely change, but other attributes, like reservations and orderHistories, accumulated with the table represent current state data that changes over time.

Its purpose is to encapsulate all table-related data, including the table's unique identifier, seating capacity, and whether it's an admin table. Keeping related data together in a dedicated class ensures clear ownership and easy access to table-specific information.

Below is the representation of the class.

Layout

The Layout class manages all restaurant tables, organizing them by ID and capacity to provide the right one for each booking. It exposes methods to add tables, get tables by ID, and find available tables for a given party size. Keeping this in a dedicated class centralizes table management and makes it easy to add new table types or sort by different criteria.

Below is the representation of the class.

OrderItem

The OrderItem class represents each item in a customer's order, linking it to a specific MenuItem to provide details like price for the bill. It also stores quantity and special instructions, allowing for flexible ordering. Keeping this in a dedicated class ensures each order item is self-contained and easy to modify or remove before being accepted. The class is created when customers place orders, using the Status enum, left to extend the ordering workflow if needed, for example, tracking preparation status.

Below is the representation of the class.

ReservationManager

The ReservationManager class handles reservation scheduling by finding available times, creating reservations, and processing cancellations. It stores all active reservations and uses a reference to the Layout to check table availability and assign reservations. This centralizes scheduling logic so party sizes, availability, and assign suitable tables for each reservation based on party size and time.

Below is the representation of the class.

Reservation

The Reservation class represents a single booking managed by the ReservationManager. Unlike, storing the party name, number of people, reservation time, and assigned table, it serves to track all details of a reservation, enabling the ReservationManager to schedule and cancel bookings efficiently.

Below is the representation of the class.

The class is constructed within ReservationManager as one of its entries, maintaining a structured and accessible set of reservation data.

Restaurant

The Restaurant class serves as the primary interface and facade for the restaurant management system. It coordinates user commands (seating operations, orders, reservations, assigning tables, processing payments, and handling checkout).

It simplifies access to these features by delegating tasks to other classes.

Design choice: The facade pattern centralizes the system's complexity, keeping the main interface clean and easy to use. This approach ensures that the system's complexity is hidden from the user, providing a simple, unified interface for interacting with the restaurant's operations.

Implementation Choice: We initialize the Restaurant as a Singleton to avoid multiple instances and ensure consistent state management. While this simplifies access to the restaurant's operations and manages its state consistently, in a production system, this might need to be replaced with dependency injection for better testability and scalability.

Below is the representation of the class.

Class Diagram of Restaurant Management System

With the core classes in place, let's move on to implement the code that brings this design to life.

Code - Restaurant Management System

In this section, we'll implement the core functionalities of the Restaurant Management System, focusing on key areas such as managing menus, table assignments, processing customer reservations, assigning waiters for bookings and walk-ins, and processing orders with billing through a table based system.

Menu

The menu class manages the restaurant's menu by storing menu items in a hashmap, using exact menu items as the key to enable fast retrieval and prevent duplicates. It provides methods to add an item with validation, retrieve a specific item by name, and access the full menu through defensive copies to maintain data integrity.

Implementation Choice: We utilize a hashmap for menu storage, ensuring efficient access to menu items based on their names. While this offers optimal performance for lookups, ensuring consistent ordering or alternatives, when displaying the menu might be considered for a more user-friendly presentation.

The definition of the menu class is given below.

```
class Menu:
    private Map<String, MenuItem> menuItems = new HashMap<>();

    // Add a new item to the menu
    public void addItem(MenuItem item) {
        menuItems.put(item.getName(), item);
    }

    public MenuItem getItem(String name) {
        return menuItems.get(name);
    }

    public Map<String, MenuItem> getMenuItems() {
        return Collections.unmodifiableMap(menuItems);
    }
```

With the menu defined, let's extend its individual items in the menu.

MenuItem

The MenuItem class represents an individual item on the menu, storing its name, description, price, and category as private final fields to ensure immutability. The getter methods provide access to these fields without allowing changes, maintaining data integrity. The class uses Lombok's `@Getter` annotation for financial data, as it provides precise control over decimal values and helps avoid rounding errors in calculations that are critical for billing and customer transactions.

The implementation of the MenuItem class is provided in the restaurant demo.

```
public class MenuItem {
    private final String name;
    private final String description;
    private final double price;
    private final Category category;

    // Constructor for initializing menu item properties
    public MenuItem(String name, String description, double price, Category category) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.category = category;
    }

    // Enumeration of possible menu item categories
    public enum Category {
        APPETIZER,
        MAIN_COURSE,
        DESSERT,
        BEVERAGE
    }
}
```

Having established the menu's contents, let's model the restaurant's tables with Table.

Table

The Table class represents a restaurant table, storing its fixed properties, such as tableNumber and capacity, while maintaining its bookings and orders by updating relevant collections. The class provides methods for its bookings to manage its reservations and orders, using Lombok's `@Getter` and `@Setter` annotations to provide prime and other access.

Below is the representation of the class.

```
class Table {
    private int tableNumber;
    private int capacity;
    private boolean isAdmin;
    private List<Reservation> reservations = new ArrayList<>();
    private List<Order> orderHistory = new ArrayList<>();

    // Constructor to initialize table with basic properties
    public Table(int tableNumber, int capacity, boolean isAdmin) {
        this.tableNumber = tableNumber;
        this.capacity = capacity;
        this.isAdmin = isAdmin;
    }

    // Calculates how many party sizes fit at different times on this table
    // Used by ReservationManager to find suitable tables
    private int[] getAvailabilityMatrix(int[] timeSlots) {
        int[] availability = new int[timeSlots.length];
        for (int i = 0; i < timeSlots.length; i++) {
            int time = timeSlots[i];
            for (Reservation r : reservations) {
                if (r.getTime().equals(time)) {
                    availability[i] -= r.getNumPeople();
                }
            }
            availability[i] += capacity;
        }
        return availability;
    }

    // Adds reservation and updates all time slots based on new reservation
    public void addReservation(Reservation reservation, int[] timeSlots) {
        int[] availabilityMatrix = getAvailabilityMatrix(timeSlots);
        reservations.add(reservation);
    }

    // Adds a single order to the table's order
    // Used when customer places an order
    public void addOrder(Order order, LocalDateTime currentTime) {
        int[] timeSlots = {currentTime};
        int[] availabilityMatrix = getAvailabilityMatrix(timeSlots);
        orderHistory.add(order);
        int remainingCapacity = availabilityMatrix[0];
        int newCapacity = remainingCapacity - order.getNumPeople();
        availabilityMatrix[0] = newCapacity;
    }

    // Removes a reservation from this table
    // Used when cancelling reservations
    public void removeReservation(Reservation reservation) {
        reservations.remove(reservation);
    }

    // Getter methods are provided by Lombok
}
```

This class uses a time-slot reservation three array to store reservations. This setup enables quick availability checks through the availability matrix. It also manages both current and future bookings by storing reservations in the reservations list. The class also tracks table attributes like capacity and admin status, ensuring flexibility for various table types.

With tables in place, let's model the customer reservation system.

Layout

The Layout class manages the seating arrangement of the entire restaurant, managing all tables to support efficient assignments for reservations and walk-ins. It organizes tables by their identifiers and provides methods to find suitable tables.

- **Tables by ID:** Stores tables in a hashmap for efficient retrieval using unique identifiers. This enables fast access to specific tables when needed.
- **Tables by Capacity:** Stored in a TreeMap (sorted hashmap) where each capacity level maps to a set of tables with that capacity. This organization helps find appropriate tables for groups by quickly accessing tables that fit a specific party size.

Below is the implementation of this class.

```
// Manages the arrangement of tables in the restaurant and their assignment
// to different party sizes
class Layout {
    private Map<Integer, Table> tables = new HashMap<>();
    private TreeMap<Integer, Set<Table>> tablesByCapacity = new TreeMap<>();

    public void addTable(int id, int tableSize) {
        Table t = new Table(id, tableSize, false);
        tables.put(id, t);
        if (!tablesByCapacity.containsKey(tableSize)) {
            tablesByCapacity.put(tableSize, new HashSet<>());
        }
        tablesByCapacity.get(tableSize).add(t);
    }

    public Table getTable(int id) {
        return tables.get(id);
    }

    public List<Table> findAvailableTables(int partySize, LocalDateTime reservationTime) {
        int minCapacity = partySize;
        int maxCapacity = partySize + 2;
        List<Table> result = new ArrayList<>();
        SortedMap<Integer, Set<Table>> subMap = tablesByCapacity.subMap(minCapacity, true, maxCapacity, true);
        for (Set<Table> tables : subMap.values()) {
            for (Table t : tables) {
                if (!t.getReservations().contains(reservationTime)) {
                    result.add(t);
                }
            }
        }
        return result;
    }
}
```

Implementation Choice: We chose TreeMap structure for tablesByCapacity to efficiently find tables within specific capacity ranges. The sorted nature of TreeMap allows quick access to relevant capacity levels, making it ideal for finding suitable tables. While it slightly increases insertion time compared to a regular HashMap, the performance benefit for capacity-based searches outweighs this minor overhead.

OrderItem

The OrderItem class defines each item ordered by a customer, storing a reference to a specific MenuItem and maintaining its current status to track its order journey from preparing to served.

The class includes methods to update the order's status:

- **markPreparing()**: Changes the status from PENDING to IN_PREPARATION
- **markServed()**: Updates status from IN_PREPARATION to READY
- **markCancelled()**: Sets status to CANCELLED

```
class OrderItem {
    // Track the current status of this specific order item
    private Status status;

    // Mark this order item as being prepared
    public void markPreparing() {
        if (status == PENDING) {
            status = IN_PREPARATION;
        }
    }

    // Mark this order item as ready to be served
    public void markServed() {
        if (status == IN_PREPARATION) {
            status = READY;
        }
    }

    // Cancel this order item if it's still pending
    public void markCancelled() {
        if (status == PENDING) {
            status = CANCELLED;
        }
    }

    // Getter methods are provided by Lombok
}
```

With orders tracked, let's connect reservations through ReservationManager.

ReservationManager

The ReservationManager manages all reservations in the restaurant, managing their scheduling, creation, and cancellation to ensure tables are assigned promptly. It serves as a central coordinator, connecting the Layout class to guests available tables and managing the complete reservation lifecycle.

Below is the implementation of this class.

```
// Manages all reservations for the restaurant.
// Handles table assignments and reservation lifecycle.
class ReservationManager {
    private Layout layout;
    private Map<Integer, Reservation> reservations = new HashMap<>();

    // Initialize with a reference to the layout
    public ReservationManager(Layout layout) {
        this.layout = layout;
    }

    // Find available time slots for a given party size and date
    public List<LocalDateTime> findAvailableSlots(int partySize, LocalDate date) {
        List<LocalDateTime> availableSlots = new ArrayList<>();
        int[] timeSlots = {10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22};
        for (int time : timeSlots) {
            LocalDateTime dt = date.atTime(time, 0);
            List<Table> availableTables = layout.findAvailableTables(partySize, dt);
            if (!availableTables.isEmpty()) {
                availableSlots.add(dt);
            }
        }
        return availableSlots;
    }

    // Create a new reservation for the customer
    public Reservation createReservation(int customerId, int partySize, LocalDateTime time) {
        int[] timeSlots = {time.getHour()};
        List<Table> availableTables = layout.findAvailableTables(partySize, time);
        if (availableTables.isEmpty()) {
            throw new IllegalStateException("No tables available");
        }
        Table selectedTable = availableTables.get(0);
        Reservation reservation = new Reservation(customerId, partySize, time, selectedTable);
        selectedTable.addReservation(reservation, timeSlots);
        reservations.put(reservation.getId(), reservation);
        return reservation;
    }

    // Cancel an existing reservation
    public void cancelReservation(int reservationId) {
        Reservation reservation = reservations.get(reservationId);
        if (reservation != null) {
            reservation.getTable().removeReservation(reservation);
            reservations.remove(reservationId);
        }
    }

    // Get all reservations for a specific date
    public List<Reservation> getReservationsForDate(LocalDate date) {
        List<Reservation> result = new ArrayList<>();
        for (Reservation r : reservations.values()) {
            if (r.getTime().toLocalDate().equals(date)) {
                result.add(r);
            }
        }
        return result;
    }

    // Get all reservations for a specific customer
    public List<Reservation> getReservationsForCustomer(int customerId) {
        List<Reservation> result = new ArrayList<>();
        for (Reservation r : reservations.values()) {
            if (r.getCustomerId().equals(customerId)) {
                result.add(r);
            }
        }
        return result;
    }

    // Find the next reservation for a party of this size
    public Reservation findNextReservationForParty(int partySize) {
        LocalDateTime now = LocalDateTime.now();
        for (int i = 0; i < 24; i++) {
            LocalDateTime checkTime = now.plusHours(i);
            List<Table> availableTables = layout.findAvailableTables(partySize, checkTime);
            if (!availableTables.isEmpty()) {
                return new Reservation(-1, partySize, checkTime, availableTables.get(0));
            }
        }
        return null;
    }

    // Mark reservation as completed
    public void markReservationAsCompleted(int reservationId) {
        Reservation r = reservations.get(reservationId);
        if (r != null) {
            r.markCompleted();
        }
    }

    // Mark reservation as no show
    public void markReservationAsNoShow(int reservationId) {
        Reservation r = reservations.get(reservationId);
        if (r != null) {
            r.markNoShow();
        }
    }
}
```

With orders tracked, let's connect reservations through ReservationManager.

Restaurant

The Restaurant class serves as the main facade for the restaurant, managing their scheduling, creation, and cancellation to ensure tables are assigned promptly. It serves as a central coordinator, connecting the Layout class to guests available tables and managing the complete reservation lifecycle.

Below is the implementation of this class.

```
// Manages all reservations for the restaurant.
// Handles table assignments and reservation lifecycle.
class Restaurant {
    private Layout layout;
    private ReservationManager reservationManager;

    // Initialize with a reference to the layout
    public Restaurant(int numberOfTables) {
        this.layout = new Layout();
        this.reservationManager = new ReservationManager(layout);
        initializeTables(numberOfTables);
        reservationManager.initializeReservations();
    }

    // Create initial tables based on the specified number
    private void initializeTables(int numberOfTables) {
        int[] capacities = {2, 4, 6, 8};
        for (int i = 1; i <= numberOfTables; i++) {
            int capacity = capacities[(i % capacities.length)];
            layout.addTable(i, capacity);
        }
    }

    // Add a new reservation for a customer
    public void addReservation(int customerId, int partySize, LocalDateTime time) {
        Reservation reservation = reservationManager.createReservation(customerId, partySize, time);
        if (reservation != null) {
            System.out.println("Reservation created successfully");
        } else {
            System.out.println("Failed to create reservation");
        }
    }

    // Cancel an existing reservation
    public void cancelReservation(int reservationId) {
        reservationManager.cancelReservation(reservationId);
        System.out.println("Reservation cancelled");
    }

    // Seat a walk-in customer at an available table
    public void seatWalkIn(int partySize, int customerId) {
        LocalDateTime now = LocalDateTime.now();
        List<Table> availableTables = layout.findAvailableTables(partySize, now);
        if (!availableTables.isEmpty) {
            Table table = availableTables.get(0);
            Reservation walkIn = new Reservation(customerId, partySize, now, table);
            table.addReservation(walkIn, new int[]{now.getHour()});
            System.out.println("Walk-in seated at table " + table.getTableNumber());
        } else {
            System.out.println("No tables available");
        }
    }

    // Add an order to a table
    public void addOrder(int tableNumber, int customerId, int menuItemId, int quantity, String specialInstructions) {
        Table table = layout.getTable(tableNumber);
        if (table != null) {
            MenuItem item = menuItems.get(menuItemId);
            if (item != null) {
                OrderItem order = new OrderItem(customerId, item, quantity, specialInstructions);
                table.addOrder(order, LocalDateTime.now());
                System.out.println("Order added to table " + tableNumber);
            } else {
                System.out.println("Menu item not found");
            }
        } else {
            System.out.println("Table not found");
        }
    }

    // Mark order as being prepared
    public void markOrderAsPreparing(int orderId) {
        OrderItem order = findOrderById(orderId);
        if (order != null) {
            order.markPreparing();
            System.out.println("Order marked as preparing");
        }
    }

    // Mark order as ready to be served
    public void markOrderAsReady(int orderId) {
        OrderItem order = findOrderById(orderId);
        if (order != null) {
            order.markServed();
            System.out.println("Order marked as ready");
        }
    }

    // Mark order as served
    public void markOrderAsServed(int orderId) {
        OrderItem order = findOrderById(orderId);
        if (order != null) {
            order.markServed();
            System.out.println("Order marked as served");
        }
    }

    // Cancel an order
    public void markOrderAsCancelled(int orderId) {
        OrderItem order = findOrderById(orderId);
        if (order != null) {
            order.markCancelled();
            System.out.println("Order cancelled");
        }
    }

    // Mark reservation as completed
    public void markReservationAsCompleted(int reservationId) {
        reservationManager.markReservationAsCompleted(reservationId);
        System.out.println("Reservation completed");
    }

    // Mark reservation as no show
    public void markReservationAsNoShow(int reservationId) {
        reservationManager.markReservationAsNoShow(reservationId);
        System.out.println("Marked as no show");
    }

    // Print the menu to console
    public void printMenu() {
        System.out.println("=== MENU ===");
        for (MenuItem item : menuItems.getMenuItems().values) {
            System.out.println(item.getName() + " - $" + item.getPrice() + " (" + item.getCategory() + ")");
            System.out.println("  " + item.getDescription());
        }
    }

    // Print available tables for a party size and date
    public void printAvailableTables(int partySize, LocalDate date) {
        List<LocalDateTime> slots = reservationManager.findAvailableSlots(partySize, date);
        System.out.println("Available slots for " + partySize + " people on " + date);
        for (LocalDateTime slot : slots) {
            System.out.println(slot.getHour() + ":00");
        }
    }

    // Print all reservations for a specific date
    public void printReservationsForDate(LocalDate date) {
        List<Reservation> reservations = reservationManager.getReservationsForDate(date);
        System.out.println("Reservations for " + date);
        for (Reservation r : reservations) {
            System.out.println("#" + r.getId() + " - " + r.getPartyName() + " (" + r.getNumPeople() + ") at " + r.getTime() + " - " + r.getStatus());
        }
    }

    // Print all reservations for a specific customer
    public void printReservationsForCustomer(int customerId) {
        List<Reservation> reservations = reservationManager.getReservationsForCustomer(customerId);
        System.out.println("Reservations for customer " + customerId);
        for (Reservation r : reservations) {
            System.out.println("#" + r.getId() + " - " + r.getDate() + " at " + r.getTime() + " - " + r.getStatus());
        }
    }

    // Print next reservation for a party of this size
    public void printNextReservationForParty(int partySize) {
        Reservation next = reservationManager.findNextReservationForParty(partySize);
        if (next != null) {
            System.out.println("Next reservation for " + partySize + " people:");
            System.out.println("#" + next.getId() + " - " + next.getDate() + " at " + next.getTime());
            System.out.println("Table " + next.getTable().getTableNumber());
        } else {
            System.out.println("No upcoming reservations");
        }
    }

    // Print all orders for a table
    public void printOrdersForTable(int tableNumber) {
        Table table = layout.getTable(tableNumber);
        if (table != null) {
            List<OrderItem> orders = table.getOrderHistory();
            System.out.println("Orders for table " + tableNumber);
            for (OrderItem order : orders) {
                System.out.println("#" + order.getId() + " - " + order.getItem().getName() + " x" + order.getQuantity() + " - " + order.getStatus());
                if (order.getSpecialInstructions() != null) {
                    System.out.println("  Special: " + order.getSpecialInstructions());
                }
            }
        }
    }

    // Print all orders in the system
    public void printAllOrders() {
        System.out.println("=== ALL ORDERS ===");
        for (Table t : layout.getAllTables().values) {
            List<OrderItem> orders = t.getOrderHistory();
            if (!orders.isEmpty) {
                System.out.println("Table " + t.getTableNumber() + ":");
                for (OrderItem order : orders) {
                    System.out.println("#" + order.getId() + " - " + order.getItem().getName() + " x" + order.getQuantity() + " - " + order.getStatus());
                }
            }
        }
    }

    // Print total revenue from all completed orders
    public void printTotalRevenue() {
        double total = 0.0;
        for (Table t : layout.getAllTables().values) {
            List<OrderItem> orders = t.getOrderHistory();
            for (OrderItem order : orders) {
                if (order.getStatus() == Status.SERVED) {
                    total += order.getItem().getPrice() * order.getQuantity();
                }
            }
        }
        System.out.println("Total revenue: $" + total);
    }
}
```

With core classes, let's connect reservations through ReservationManager.

```
    table.addReservation(reservation);
    reservations.add(reservation);
    return reservation;
  }

  // Removes an existing reservation
  public void removeReservation(
      String partyName, int partySize, LocalDateTime reservationTime) {
    // Find matching reservation before removing it
    for (Reservation reservation : new HashSet<>(reservations)) {
      if (reservation.getPartyName().equals(partyName)
          && reservation.getPartySize() == partySize
          && reservation.getPartyName().equals(partyName)) {
        // Clear the reservation from the table first
        Table table = reservation.getAssignedTable();
        table.removeReservation(reservationTime);
        // Then remove from the reservation collection
        reservations.remove(reservation);
        return;
      }
    }
  }

  // getter methods are omitted for brevity
}
```

- **findAvailableTimeSlots**: This method identifies available reservation times within a specified range. It examines each hour in the range, assuming a one-hour duration per reservation.
- **createReservation**: This method creates a reservation for a given time, party size, and party name. It uses Layout.findAvailableTable() to confirm a table's availability. If a table is found, it generates a new Reservation object, adds it to the reservations set, and updates the table's booking status.
- **removeReservation**: This method cancels a reservation based on its time, party size, and party name. It searches the reservations set to locate the matching entry, removes it, and clears the table's booking if found.

Implementation Choice: We chose a Set to store Reservation objects because its unique entry enforcement prevents duplicate bookings, aligning with the need to manage reservations accurately. An alternative could use a List, which allows simpler iteration but risks duplicates unless additional checks are added, or a Map with time-based keys, which could speed up lookups but complicate removal by requiring keys that combine multiple fields, like time, party size, and name, less suited for the system's focus on reservation uniqueness.

Next, let's define the reservation entries with Reservation.

Reservation

The Reservation class is a simple, immutable entry class that stores essential reservation details, including the party name, number of people, reservation time, and assigned table. It serves as the foundational unit for the ReservationManager class, holding the data needed to manage bookings efficiently.

```
  // Represents a reservation made at the restaurant for a specific party, time and table
  public class Reservation {
    private final String partyName;
    private final int partySize;
    private final LocalDateTime time;
    private final Table assignedTable;

    public Reservation(
        String partyName, int partySize, LocalDateTime time, Table assignedTable) {
      this.partyName = partyName;
      this.partySize = partySize;
      this.time = time;
      this.assignedTable = assignedTable;
    }

    // getter methods are omitted for brevity
  }
```

Finally, let's unify these components in the Restaurant.

Restaurant

The Restaurant class acts as the central interface for the restaurant management system, offering methods to book reservations, seat walk-in parties, place orders, and compute bills. It follows the facade design pattern to unify these features, delegating reservation and walk-in seating tasks to ReservationManager, which uses Layout to find available tables, order placement, and billing to Table, which uses Menu to access items.

Below is the implementation of this class.

```
  // Main restaurant class that manages reservations, orders, and tables
  public class Restaurant {
    private final String name;
    private final Menu menu;
    private final Layout layout;
    private final ReservationManager reservationManager;

    public Restaurant(String name, Menu menu, Layout layout) {
      this.name = name;
      this.menu = menu;
      this.layout = layout;
      this.reservationManager = new ReservationManager(layout);
    }

    // Finds possible reservation times within a time range for a party of specified size
    public List<LocalTime[]> findAvailableTimeSlots(
        LocalDateTime rangeStart, LocalDateTime rangeEnd, int partySize) {
      return reservationManager.findAvailableTimeSlots(rangeStart, rangeEnd, partySize);
    }

    // Creates a reservation for a party at the specified time
    public Reservation createReservation(
        String partyName, int partySize, LocalDateTime time) {
      return reservationManager.createReservation(partyName, partySize, time);
    }

    // Removes an existing reservation
    public void removeReservation(
        String partyName, int partySize, LocalDateTime reservationTime) {
      reservationManager.removeReservation(partyName, partySize, reservationTime);
    }

    // Creates a reservation for a party without prior reservation
    public Reservation createWalkInReservation(String partyName, int partySize) {
      return reservationManager.createReservation(partyName, partySize, LocalDateTime.now());
    }

    // Adds an item to a table's order
    public void orderItem(Table table, MenuItem item) {
      table.addOrder(item);
    }

    // Removes an item from a table's order
    public void cancelItem(Table table, MenuItem item) {
      table.removeOrder(item);
    }

    // Calculates the bill amount for a table
    public BigDecimal calculateTableBill(Table table) {
      return table.calculateBillAmount();
    }

    // getter methods are omitted for brevity
  }
```

- **Reservation methods**: The Restaurant class includes findAvailableTimeSlots to list available times, createScheduledReservation to book reservations, and removeReservation to cancel them, each delegating to ReservationManager for scheduling and management.
- **Walk-in seating**: The createWalkInReservation method assigns tables to walk-in parties, delegating to ReservationManager, which uses Layout to locate available tables.
- **Order management**: The orderItem method adds items to a table's order, and cancelItem removes them, both delegating to Table methods addOrder and removeOrder for order processing.
- **Checkout**: The calculateTableBill method computes a table's total cost, delegating to Table.calculateBillAmount for billing calculations.

Implementation Choice: We designed the Restaurant as a facade without its data structures. It delegates all operations to Menu, Layout, ReservationManager, and Table. This approach maintains a lightweight structure that simplifies system access. An alternative could have Restaurant store reservations or orders internally using a Map or List. That design would increase complexity and reduce modularity by centralizing responsibilities.

Having built the core system, let's explore an enhancement in the deep dive to extend its capabilities.

Deep Dive Topics

In this section, we'll explore an enhancement to the Restaurant Management System by improving order handling during peak times. We'll focus on adding a centralized order queue tracking mechanism to streamline kitchen coordination, ensure scalability, and maintain consistency with the system's modular design.

Order queue tracking

Consider a high-traffic scenario, such as a busy Friday evening at the restaurant, where a significant volume of orders is received, staff work diligently to communicate these to the kitchen, and cancellations accumulate. In the existing system, the Table class directly governs the status of Orderitem instances (e.g., through methods like sendToKitchen() and deliverToCustomer()). However, this decentralized structure lacks a cohesive overview of order progression across all tables. Consequently, staff face challenges in prioritizing time-sensitive orders, coordinating kitchen delays, or verifying overall progress without individually inspecting each table's state. This approach introduces risks of inconsistency and undermines effective coordination during periods of elevated demand.

To address these limitations, we propose an enhancement by introducing a centralized OrderManager class responsible for queuing and processing order-related actions. Let's take a closer look.

Implementation steps:

To implement this enhancement effectively, we'll follow these steps:

Step 1: Define a Command Interface for Order Actions: The staff needs a consistent way to issue actions like sending an order to the kitchen or cancelling it, improving reliability over direct calls. We define an interface called OrderCommand with a single method, execute(), which concrete classes will implement to perform their tasks.

Here's the code for the OrderCommand interface:

```
public interface OrderCommand {
    void execute();
}
```

Step 2: Implement Concrete Command Classes: To give staff a flexible way to manage distinct order actions and prepare them for centralized queuing, we build specific classes for each task, unlike the original system, where Table directly updated Orderitem statuses.

- The SendToKitchenCommand class takes an Orderitem and, when executed, calls sendToKitchen() to update its status to SENT_TO_KITCHEN.
- The DeliverCommand class uses deliverToCustomer() to change the Orderitem's status to DELIVERED, signaling completion.
- The CancelCommand class invokes cancel() to set the status to CANCELED if the item hasn't been delivered, marking it as voided.

Here's the code for the action classes:

```
// Command that handles sending order items to the Kitchen
public class SendToKitchenCommand implements OrderCommand {
    private final Orderitem orderItem;

    public SendToKitchenCommand(Orderitem orderItem) {
        this.orderItem = orderItem;
    }

    @Override
    public void execute() {
        orderItem.sendToKitchen();
    }
}

// Command that handles delivery of order items

public class DeliverCommand implements OrderCommand {
    private final Orderitem orderItem;

    public DeliverCommand(Orderitem orderItem) {
        this.orderItem = orderItem;
    }

    @Override
    public void execute() {
        orderItem.deliverToCustomer();
    }
} // Command that handles cancellations of order items

public class CancelCommand implements OrderCommand {
    private final Orderitem orderItem;

    public CancelCommand(Orderitem orderItem) {
        this.orderItem = orderItem;
    }

    @Override
    public void execute() {
        orderItem.cancel();
    }
}
```

Step 3: Introduce the OrderManager Class: To handle these actions efficiently, we introduce an OrderManager class. This class maintains a list of OrderCommand objects, allowing us to add commands with addCommand() as orders are placed. The executeCommands() method processes all queued commands in sequence and clears the list afterward, ensuring orders are managed in an organized way without directly altering the Table's Orderitems.

Here's the code for the OrderManager class:

```
public class OrderManager {
    private Final List<OrderCommand> commandQueue = new ArrayList<>();

    // Adds a command to the queue for later execution
    public void addCommand(OrderCommand command) {
        commandQueue.add(command);
    }

    // Executes all commands in the queue and clears it
    public void executeCommands() {
        for (OrderCommand command : commandQueue) {
            command.execute();
        }
        commandQueue.clear();
    }
}
```

Step 4: Integrate with the Restaurant Class: To connect this system to staff actions, we update the Restaurant class to use OrderManager for order handling. When staff place an order, orderItem adds it to the table and queues a SendToKitchenCommand to send it to the kitchen. Similarly, cancelItem and deliverItem queue their respective commands, ensuring all actions flow through the centralized system for consistent tracking and execution.

Here's the essential code for integrating OrderManager into the Restaurant:

```
public class Restaurant {
    // ... fields unchanged ...
    private Final OrderManager orderManager;

    public Restaurant(String name, Menu menu, Layout layout) {
        // ... fields unchanged
        this.orderManager = new OrderManager();
    }

    // Adds an item to a table's order and sends it to the kitchen
    public void orderItem(Table table, MenuItem item) {
        table.addOrder(item);
        // Get the last added order item
        List<Orderitem> orderItems = table.getOrderedItems();
        if (!orderItems.isEmpty() && !orderItems.isEmpty()) {
            Orderitem lastOrder = orderItems.get(orderItems.size() - 1);
            OrderCommand sendToKitchen = new SendToKitchenCommand(lastOrder);
            orderManager.addCommand(sendToKitchen);
            orderManager.executeCommands();
        }
    }

    // Removes an item from a table's order and cancels it
    public void cancelItem(Table table, MenuItem item) {
        List<Orderitem> orderItems = table.getOrderedItems();
        if (!orderItems.isEmpty() && !orderItems.isEmpty()) {
            Orderitem lastOrder = orderItems.get(orderItems.size() - 1);
            OrderCommand cancelOrder = new CancelCommand(lastOrder);
            orderManager.addCommand(cancelOrder);
            orderManager.executeCommands();
            table.removeOrder(item);
        }
    }

    // Delivers an item to the customer
    public void deliverItem(Table table, MenuItem item) {
        List<Orderitem> orderItems = table.getOrderedItems();
        if (!orderItems.isEmpty() && !orderItems.isEmpty()) {
            Orderitem lastOrder = orderItems.get(orderItems.size() - 1);
            OrderCommand deliverOrder = new DeliverCommand(lastOrder);
            orderManager.addCommand(deliverOrder);
            orderManager.executeCommands();
        }
    }
    // ... other methods unchanged ...
}
```

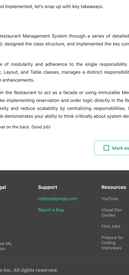
What we've just implemented follows a well-known software design pattern called the **Command Pattern**.

> **Definition:** The Command is a behavioral design pattern that encapsulates a request as an independent object, containing all the details needed to carry it out. This encapsulation allows you to treat requests as parameters for methods, delay or schedule their execution.

In this pattern:

- **Command**: The OrderCommand Interface and its implementations (SendToKitchenCommand, DeliverCommand, CancelCommand) define each action as an object, executed via execute().
- **Invoker**: The OrderManager class queues these commands (objects and triggers their execution, decoupling the Table from direct state changes.
- **Receiver**: The Orderitem class, with its existing Status enum and methods like sendToKitchen, receives and processes the commands, updating its state to reflect actions like preparation or delivery.

The UML diagram below illustrates this structure.



Command pattern

With our restaurant management system designed and implemented, let's wrap up with key takeaways.

Wrap Up

In this chapter, we gathered requirements for the Restaurant Management System through a series of detailed questions and answers. We then identified the core objects involved, designed the class structure, and implemented the key components of the system.

A key takeaway from this design is the importance of modularity and adherence to the single responsibility principle. Each component, such as the Menu, ReservationManager, Layout, and Table classes, manages a distinct responsibility, ensuring the system remains maintainable and adaptable for future enhancements.

Our design choices, such as delegating operations in the Restaurant to act as a facade or using immutable Menuitem objects, prioritize flexibility and consistency. An alternative, like implementing reservation and order logic directly in the Restaurant class, may simplify the design but could increase complexity and reduce scalability by centralizing responsibilities. In an interview, revisiting these decisions and explaining their rationale demonstrates your ability to think critically about system design.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask AlgoMaster

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.